GIROFEEDS · MAGENTO 2 EXTENSION

Installation & Configuration Guide

How to install, configure and connect the **Girofeeds Product API** extension so your Magento 2 catalog syncs with the Girofeeds platform.

Girofeeds_ProductApi | v1.23.0 | Magento 2.4.x | PHP 8.1+

Composer package: girofeeds/magento2-girofeeds
License: BSD-2-Clause

Contents

1. Introduction
2. Requirements
3. Installation
4. Post-installation Commands
5. Configuration
6. Connecting to Girofeeds
7. Verifying the Integration
8. Troubleshooting

1. Introduction

The Girofeeds Product API extension turns your Magento 2 store into a connector for the Girofeeds catalog platform.

Once installed, the extension does two things:

- **Exposes a product feed.** It publishes your catalog as an authenticated JSON feed that Girofeeds reads to import products, images, prices, stock and attributes. The feed is paginated and highly configurable (attribute mapping, tax handling, product-type behaviour, filters, inventory data and optional per-product order counts).
- **Receives updates back from Girofeeds.** A secure endpoint lets the Girofeeds platform push product changes (for example price or stock updates) back into your Magento store.

Every request is authenticated with a single API key that you generate inside Magento and paste into Girofeeds. Requests are rate-limited and logged, and all endpoints share the same key so the connection is easy to manage.

> **Where it fits**
> Girofeeds (SaaS) ⟷ **Girofeeds Product API (this extension)** ⟷ your Magento 2 catalog. The extension is the bridge: it reads products out and writes updates back in.

2. Requirements

These are the minimum requirements declared by the extension (`composer.json`) and verified against the module code:

Magento	Magento 2 / Adobe Commerce **2.4.x** (`magento/framework >= 102.0.3`)
PHP	**8.1** or higher
PHP extensions	`ext-json`
Access	SSH / CLI access to run `bin/magento`, plus Admin access to *Stores → Configuration*
Composer	Composer 2 (for the Marketplace and Composer installation methods)
Extension	Package `girofeeds/magento2-girofeeds`, module name `Girofeeds_ProductApi`, version **1.23.0**

Before you start

Take a backup and run the installation in a staging environment first. If your store is in *production* mode, you will also redeploy static content and recompile — plan a short maintenance window.

3. Installation

Choose **one** of the three methods below. Option 1 (Adobe Commerce Marketplace) is the recommended path for production stores; Options 2 and 3 are convenient for development or when installing directly from source.

Option 1 — Install from the Adobe Commerce Marketplace (recommended)

The Marketplace method installs the extension through Composer using your Adobe Commerce (Magento) account authentication keys.

1. **Get the extension.** Sign in to the Adobe Commerce Marketplace, open the *Girofeeds Product API* listing and purchase / add it to your account. Free extensions are added the same way — you still complete "checkout" to attach the component to your account.
2. **Get your access keys.** In the Marketplace, go to *My Profile → Access Keys* (Marketplace API keys). Use the *public key* as your Composer username and the *private key* as your password. If you have never authenticated `repo.magento.com` on this server, add the keys:

   ```
   composer config --global http-basic.repo.magento.com <public-key> <private-key>
   ```

3. **Require the extension** from your Magento project root:

   ```
   composer require girofeeds/magento2-girofeeds
   ```

4. Continue with the Post-installation Commands below.

Tip

You can also start the purchase from your store Admin via *System → Web Setup Wizard → Component Manager / Extension Manager*, which links back to your Marketplace account. The underlying install is the same `composer require` shown above.

Option 2 — Install with Composer (direct package)

If you distribute the package through your own Composer repository (or once it is published), require it by name from the Magento project root:

```
composer require girofeeds/magento2-girofeeds
bin/magento module:enable Girofeeds_ProductApi
bin/magento setup:upgrade
bin/magento setup:di:compile
bin/magento cache:flush
```

If you serve the package from a private / path repository, add it to the project `composer.json` `repositories` section first, then run the `composer require` above.

Option 3 — Manual installation in `app/code`

Copy the extension source directly into the Magento codebase. This is the method most used for development and when installing from a Git checkout or ZIP.

1. Create the module directory and copy the files into it:

   ```
   cd <magento-root>
   mkdir -p app/code/Girofeeds/ProductApi
   # copy the extension source (git clone, scp, or unzip) into:
   #   app/code/Girofeeds/ProductApi/
   ```

2. Set correct ownership and permissions (replace `www-data` with your web-server user):

   ```
   find app/code/Girofeeds -type f -exec chmod 644 {} \;
   find app/code/Girofeeds -type d -exec chmod 755 {} \;
   chown -R www-data:www-data app/code/Girofeeds
   ```

3. Continue with the Post-installation Commands below.

4. Post-installation Commands

Run these from the Magento project root after any of the installation methods above. They enable the module, apply its schema, compile dependencies, generate the API key and clear caches.

```
bin/magento module:enable Girofeeds_ProductApi
bin/magento setup:upgrade
bin/magento setup:di:compile
bin/magento girofeeds:token:generate
bin/magento cache:flush
```

On a **production-mode** store, also redeploy static content:

```
bin/magento setup:static-content:deploy
bin/magento cache:flush
```

WHAT EACH COMMAND DOES

Command	Purpose
`module:enable Girofeeds_ProductApi`	Registers and enables the module.
`setup:upgrade`	Applies the module's DB schema (request-log table) and marks it installed.
`setup:di:compile`	Compiles dependency injection (required in production mode).
`girofeeds:token:generate`	Generates the API key used to authenticate every Girofeeds request.
`cache:flush`	Clears Magento caches so the new configuration is picked up.

EXTENSION CLI COMMANDS

Command	Description
`bin/magento girofeeds:token:generate`	Generate the API key. Fails if a key already exists (to avoid breaking a live connection).
`bin/magento girofeeds:token:generate --force=1`	Regenerate the API key, replacing the existing one. You must re-enter the new key in Girofeeds afterwards.
`bin/magento girofeeds:selftest`	Runs a self-test: checks the module is enabled, an API key is set, and a product collection can be built. Also prints the module and Magento versions.

Note on `--force`

Regeneration requires `--force=1` (with the value). Passing `--force` alone is treated as "not forcing" and the command will report that a key already exists.

CONFIRM THE MODULE IS ACTIVE

```
bin/magento module:status Girofeeds_ProductApi
# expected: Module is enabled
```

5. Configuration

All settings live under a dedicated **Girofeeds** tab in the Magento admin. Open:

Stores → Configuration → Girofeeds — which contains two sections: *General Settings* and *Product Feed*.

There is also a direct shortcut at **Stores → Girofeeds Product API** that opens the General Settings section.

Scope

Most settings can be set per *website* and per *store view*, so different store views can expose different feeds. Set the scope selector (top-left of the configuration page) before editing if you need store-specific values.

5.1 General Settings

CONNECTION INFO

This group is the bridge to the Girofeeds platform. Its values are read-only in Magento and are meant to be copied into Girofeeds.

Field	What it is
Store Domain	The base URL of this store. Copy it into the *Store Domain* field in Girofeeds.
API Key	The authentication key generated by `girofeeds:token:generate`. Copy it into the *API Key* field in Girofeeds. To rotate it, run `bin/magento girofeeds:token:generate --force=1`.
Module Version	The installed extension version. Copy it into the *Module Version* field in Girofeeds.
Store Views	The list of available store-view codes. Use the code you want in the *Store View* field in Girofeeds.

GENERAL

Field	Description	Default
Enable Module	Master switch. When off, every API request returns "Module is not enabled".	Yes
Products Per Page	Maximum number of products returned per feed request (pagination page size).	100
Include Disabled Products	Whether disabled products are included in the feed.	No
Include Out of Stock	Whether out-of-stock products are included in the feed.	Yes

DEBUG & LOGGING

Field	Description	Default
Debug Mode	When enabled, writes debug records to `var/log/girofeeds/debug.log`. Errors are always written to `var/log/girofeeds/error.log` regardless of this setting.	No

5.2 Product Feed

This section controls exactly what the feed contains and how each product type is exported. Defaults are sensible for most catalogs; adjust as needed.

PRODUCT DATA SETTINGS

Map your Magento attributes to the standard feed fields. Defaults are shown in parentheses.

Feed field	Default attribute
Name	`name`
Description	`description`
Short Description	`short_description`
Brand	`manufacturer`
EAN / GTIN	*(unset — select your barcode attribute)*
SKU	`sku`
Color / Size / Material / Gender	`color` / `size` / `material` / `gender`

IMAGE, PRICE & PRODUCT TYPES

Group	Key options
Image Settings	Image Source, Main Image Role (which role is the main image; others become additional images), Include Hidden Images.
Price Settings	Tax Calculation and Include Both Tax Variants (export prices both with and without tax). Both default to enabled.
Product Types	Per type (Configurable, Bundle, Grouped): choose whether to export the parent, the simples, or both; and optional fallbacks — use parent URL / image for simples, parent-data fallback fields, and (bundles) stock calculated from associated simples. Defaults: configurable → export simples; bundle & grouped → export parent.

ORDER COUNT, EXTRA FIELDS, FILTERS & ADVANCED

Group	What it controls
Order Count in Feed	Optionally add per-product order-count fields (`orders_1d` , `orders_7d` , `orders_30d` , `orders_90d` , `orders_365d`) and choose which order statuses to count. Disabled by default.
Extra Attribute Mappings	Add custom attribute → feed-field mappings to enrich the product data.
Filter Settings	Include/exclude products by visibility (default: Catalog and Catalog+Search), by category, by stock, plus advanced condition-based filters.
Advanced Options	Inventory data fields (defaults `min_sale_qty` , `backorders`), Force NON-MSI (legacy stock, only if you don't use MSI), Include Inventory Sources, and Content Staging (*Use Row ID* for Adobe Commerce when parent/child relations are out of sync).
Feed Overview	A read-only summary block of the configured feeds.

Minimum to go live

In practice you only need to (1) confirm *Enable Module* = Yes, (2) generate the API key, and (3) copy the four Connection Info values into Girofeeds. Everything else has working defaults.

6. Connecting to Girofeeds

Register this Magento store as a source (provider) inside the Girofeeds platform using the values from *Connection Info*.

1. In Magento, open **Stores** → **Configuration** → **Girofeeds** → **General Settings** → **Connection Info** and keep it open — you will copy four values from here.
2. In the Girofeeds application, add a new **Magento** store / provider.
3. Fill in the Girofeeds connection form:

Girofeeds field	Value from Magento
Store Domain	The *Store Domain* value (your store base URL).
API Key	The *API Key* value.
Module Version	The *Module Version* value.
Store View	The store-view code you want to export, taken from *Store Views*.

4. Save the connection in Girofeeds. Girofeeds will call the feed endpoint with this key to import your catalog.

> **Keep the key secret**
> The API key authenticates every request. Treat it like a password. If it is ever exposed, rotate it with `bin/magento girofeeds:token:generate --force=1` and update it in Girofeeds.

7. Verifying the Integration

Confirm the connection from both the command line and the browser before relying on it.

FROM THE SERVER (CLI)

```
bin/magento girofeeds:selftest
```

A healthy store prints three `[PASS]` lines (module enabled, API key set, product collection built) plus the module and Magento versions.

FROM A BROWSER OR CURL

Replace `YOUR_API_KEY` with the key from Connection Info and `STORE_ID` with the numeric store-view id.

Check	Endpoint
Module test	`GET /girofeeds/test/index?key=YOUR_API_KEY`
Product count	`GET /girofeeds/count/index?key=YOUR_API_KEY`
Product feed	`GET /girofeeds/feed/json?id=STORE_ID&token=YOUR_API_KEY`

```
# Test endpoint
curl "https://your-store.com/girofeeds/test/index?key=YOUR_API_KEY"

# Product count
curl "https://your-store.com/girofeeds/count/index?key=YOUR_API_KEY"

# First page of the feed (store view id = 1)
curl "https://your-store.com/girofeeds/feed/json?id=1&token=YOUR_API_KEY"
```

Parameter names

The *feed* endpoint expects `id` (store id) and `token` (the API key). The *test*, *count* and *updateproduct* endpoints expect the API key in `key`. Both refer to the same generated API key.

Equivalent Magento REST routes also exist for integrations that prefer them: `GET /rest/V1/girofeeds/test`, `GET /rest/V1/girofeeds/count`, `GET /rest/V1/girofeeds/feed`, and `POST /rest/V1/girofeeds/updateproduct`.

8. Troubleshooting

Symptom	Likely cause & fix
"Invalid API key" (HTTP 401)	The key sent does not match the one stored in Magento. Re-copy the *API Key* from Connection Info into Girofeeds. If you recently regenerated it, update Girofeeds with the new value.
"Missing API key" (HTTP 401)	No key was sent. Ensure `token=` (feed) or `key=` (test/count/update) is present in the request, and that a key exists — run `bin/magento girofeeds:token:generate`.
"Module is not enabled" (HTTP 403)	The module master switch is off for this scope. Set *Enable Module* = Yes under General Settings (check the store-view scope), then `bin/magento cache:flush`. Also confirm the module itself is enabled: `bin/magento module:status Girofeeds_ProductApi`.
"Store ID and token are required" (feed)	The feed call is missing `id` and/or `token`. Include both, e.g. `...?id=1&token=YOUR_API_KEY`.
"Rate limit exceeded" (HTTP 429)	More than the allowed requests within a 60-second window (60 by default). Slow the request rate; the window resets automatically after 60 seconds.
"Only POST method is allowed"	The update-product endpoint accepts `POST` only. Send updates as POST to `/girofeeds/updateproduct/index?key=YOUR_API_KEY`.
Old data / changes not reflected	Stale cache or config. Run `bin/magento cache:flush`. Endpoints themselves send no-cache headers, so persistent staleness usually points to Magento full-page/config cache.
"Class not found" after install	DI not compiled. Run `bin/magento setup:di:compile` and `composer dump-autoload`.
Permission / write errors	Fix ownership and modes (replace `www-data` with your web user): `find app/code/Girofeeds -type f -exec chmod 644 {} \;` `find app/code/Girofeeds -type d -exec chmod 755 {} \;` `chown -R www-data:www-data app/code/Girofeeds`
Need to inspect what happened	Enable *Debug Mode* and read `var/log/girofeeds/debug.log`; errors are always in `var/log/girofeeds/error.log`. Each request is logged with endpoint, hashed key, IP, HTTP status and elapsed time.

Quick health check

When in doubt, run `bin/magento girofeeds:selftest` first — it isolates whether the problem is the module, the API key, or the catalog before you look at the network layer.

Girofeeds Product API for Magento 2 · Extension version 1.23.0 · Package girofeeds/magento2-girofeeds · This guide reflects the settings exposed by the extension's admin configuration (Stores → Configuration → Girofeeds).